

Library Installation & Environment Setup

```
# Installing LangChain, Groq integration, Vector Store (FAISS), and PDF processing tools
!pip install -q langchain langchain-groq langchain-community faiss-cpu pypdf tiktoken sentence-transformers
```

Document Loading and Text Splitting

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Replace 'your_document.pdf' with the actual name of the file you uploaded
file_path = "/content/Py-tutorial.pdf"

# Load the PDF
loader = PyPDFLoader(file_path)
data = loader.load()

# Split the text into manageable chunks (1000 characters each)
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
chunks = text_splitter.split_documents(data)

print(f"Successfully split the document into {len(chunks)} chunks.")
```

Successfully split the document into 496 chunks.

Creating the Vector Store

```
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS

# Heading: Generating Embeddings and Building FAISS Vector Index

# Initialize the embedding model (this downloads a small model to Colab)
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

# Create the vector store from our PDF chunks
vector_db = FAISS.from_documents(chunks, embeddings)

# Save the vector store locally so we can use it in our Streamlit app later
vector_db.save_local("faiss_index")

print("Vector store created and saved successfully!")
```

```
/tmp/ipykernel_16539/405263284.py:7: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain
  embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
Loading weights: 100% 103/103 [00:00<00:00, 324.28it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Vector store created and saved successfully!
```

Groq LLM Configuration and Retrieval Chain

```
# Heading: Fixing Module Dependencies
!pip install -q -U langchain langchain-community langchain-groq faiss-cpu pypdf sentence-transformers
```

```
# Force-updating the specific community and groq packages to resolve the missing 'chains' module
!pip install -U -q langchain langchain-community langchain-groq
```

```
# Installing the classic package which now contains the 'chains' module
!pip install -q -U langchain-classic
```

```
import os
from langchain_groq import ChatGroq
from langchain_classic.chains import RetrievalQA

# Set your API Key
os.environ["GROQ_API_KEY"] = "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"

# Initialize the Groq LLM (Llama 3 is a great choice for speed)
llm = ChatGroq(model_name="llama3-8b-8192-8192", temperature=0)

# Verify the imports
print("Imports successful using langchain-classic!")
```

```
Imports successful using langchain-classic!
```

Setting Up Conversational Memory

```
# We must use the classic path for memory to match our other imports
from langchain_classic.memory import ConversationBufferMemory

# This object will store our chat history
memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)

print("Memory buffer initialized!")
```

```
Memory buffer initialized!
```

Building the Conversational RAG Chain

```
from langchain_classic.chains import ConversationalRetrievalChain

# This connects the Groq LLM, your Vector Store (retriever), and the memory we just created
qa_conversation = ConversationalRetrievalChain.from_llm(
    llm=llm,
    retriever=vector_db.as_retriever(),
    memory=memory
)

print("Conversational RAG Chain is ready!")
```

```
Conversational RAG Chain is ready!
```

Evaluating Memory and Retrieval Context

```
# Heading: Updating to Supported Llama 3.1 Model

# Llama 3.1 is the current replacement for the decommissioned Llama 3
llm = ChatGroq(model_name="llama-3.1-8b-instant", temperature=0)

# Re-linking the chain with the new supported model
qa_conversation = ConversationalRetrievalChain.from_llm(
    llm=llm,
    retriever=vector_db.as_retriever(),
    memory=memory
)

print("Model updated to Llama 3.1! Ready for testing.")
```

```
Model updated to Llama 3.1! Ready for testing.
```

```
# Question 1: Testing Retrieval
question_1 = "What are the main findings or topics discussed in this document?"
```

```
result_1 = qa_conversation.invoke({"question": question_1})

print(f"User: {question_1}")
print(f"AI: {result_1['answer']}")

print("-" * 30)

# Question 2: Testing Memory (Context Awareness)
# Notice we don't mention the document name; we just say "it"
question_2 = "Can you summarize the second point mentioned in it?"
result_2 = qa_conversation.invoke({"question": question_2})

print(f"User: {question_2}")
print(f"AI: {result_2['answer']}")
```

What are the main findings or topics discussed in this document?

Based on the provided context, the main findings or topics discussed in this document appear to be related to the structure and presentation of the tutorial, including the use of perlinks, footnote references, and labeled exercises to encourage presentation of information in a logical and contextual manner, with complexity and intricacy delayed until later in the tutorial. Other findings include the inclusion of features to address common errors and misconceptions, such as providing additional information to head off or react to, the use of referencing and navigation tools, including a Table of Contents, easy jumping to chosen text, cross-references, and the provision of multimedia resources, such as flash video segments, to support different learning styles. The use of a specific markup language, HTML, to produce human-readable files, and the syntax and structure of HTML tags.

Therefore, the document appears to be discussing the design and presentation of an educational resource, with a focus on how to effectively present information in a logical and contextual manner.

Can you summarize the second point mentioned in it?

The second point mentioned in the main findings or topics discussed in this document is:

Information is introduced in an order that gives you what you need as soon as possible. The information is presented in context. Complexity is introduced gradually, with simpler concepts first, and more complex concepts later.

Exporting Vector Index for Deployment

```
# Save the FAISS index (the searchable version of your PDF) to a local folder
vector_db.save_local("faiss_index_store")

print("FAISS index saved! Please download the 'faiss_index_store' folder from the Colab sidebar.")
```

FAISS index saved! Please download the 'faiss_index_store' folder from the Colab sidebar.